VITYARTI-PROJECT

2025-26

Student Performance Analyzer

Course code-CSA2001

SUBMITTED BY: KARTAVYA

REG. NO.: 25BAI11518

[Computer science with AL and ML]

INTRODUCTION

In today's academic environment, analyzing student performance is important for identifying strengths and weaknesses. However, raw marks in spreadsheets are often difficult to interpret. This project, **Student Performance Analyzer**, aims to simplify this process by using Python to analyze and visualize student data in a meaningful way.

The project focuses on transforming raw marks into useful insights through calculations and graphical representations, making it easier to understand overall and subject-wise performance.

Problem Statement

Students and teachers often struggle to:

- Identify top-performing students quickly
- Understand subject-wise performance trends
- Analyze individual student progress

Manual analysis is time-consuming and inefficient, especially with larger datasets.

Objective

The main objectives of this project are:

- To analyze student marks using Python
- To calculate total and average performance
- To identify top performers
- To generate visual graphs for better understanding
- To provide an easy-to-use command-line tool

Methodology / Approach

The project follows a structured approach:

1. Data Collection
   Student data is stored in a CSV file (students.csv).
2. Data Processing
   Using the pandas library:
   - Total marks are calculated
   - Average marks are computed
3. Analysis
   - Top performer is identified
   - Subject-wise averages are calculated
4. Visualization
   Using matplotlib, the following graphs are generated:
   - Bar chart for average marks of students
   - Bar chart for subject-wise averages
   - Line graph for individual student performance

5. User Interaction
   The user can input a student's name to view detailed performance.

Tools & Technologies Used

- Python – Core programming language
- pandas – Data analysis and manipulation
- matplotlib – Data visualization
- CSV File – Data storage

Dataset Description

The dataset is stored in a CSV file named students.csv.

Important Note:

If any changes are required (add/remove/update students), the user must manually edit the CSV file.

Given data:

```
Name, Maths, Physics, Chemistry, English, CS
Rahul,85,78,82,74,88
Anita,92,88,91,85,90
Vikram,70,65,72,68,75
Sneha,88,90,85,87,89
Amit,60,55,58,62,65
```

Program code

```
import pandas as pd
import matplotlib.pyplot as plt

#you can add or remove student data use csv file (attached with file)


df = pd.read_csv("students.csv")

# Calculate total and average
df["Total"] = df.iloc[:, 1:].sum(axis=1)
df["Average"] = df["Total"] / 5

print("\nStudent Performance Summary:\n")
print(df[["Name", "Total", "Average"]])
topper = df.loc[df["Total"].idxmax()]
print("-----------------------------------------")
print("\nTop Performer:")
print(topper["Name"], "-", topper["Total"])
print("-----------------------------------------")

# --------------
plt.figure()
plt.bar(df["Name"], df["Average"])
plt.title("Average Marks of Students")
plt.xlabel("Students")
plt.ylabel("Average Marks")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# ---------------
subject_avg = df.iloc[:, 1:6].mean()

plt.figure()
subject_avg.plot(kind="bar")
plt.title("Subject-wise Average Marks")
plt.xlabel("Subjects")
plt.ylabel("Average Marks")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

```
# -------------
student_name = input("\nEnter student name for detailed graph: ")

student_data = df[df["Name"].str.lower() == student_name.lower()]

if not student_data.empty:
    marks = student_data.iloc[0, 1:6]

    plt.figure()
    marks.plot(kind="line", marker='o')
    plt.title(f"{student_name}'s Performance")
    plt.xlabel("Subjects")
    plt.ylabel("Marks")
    plt.grid()
    plt.show()
else:
    print("Student not found!")
```

Results & Output

The program produces:

- A summary table showing total and average marks
- Name of the top performer
- Graphical outputs:
    - Student average comparison (bar graph)
    - Subject-wise performance (bar graph)
    - Individual performance trend (line graph)

These outputs make it easier to interpret academic data visually.

Challenges Faced

- Handling CSV data correctly
- Ensuring accurate calculations
- Displaying clean and readable graphs
- Managing user input errors (invalid names)

Learning Outcomes

Through this project, I learned:

- How to work with real-world datasets
- Data analysis using pandas
- Data visualization using matplotlib
- Writing structured and modular Python code
- Importance of problem-solving and logical thinking

Conclusion

The **Student Performance Analyzer** successfully demonstrates how Python can be used to solve real-world problems without complex machine learning models. It provides meaningful insights into student performance using simple calculations and effective visualizations.

This project highlights the importance of clarity, usability, and practical application of programming concepts.

REFERENCES

1. Python Official Documentation
https://docs.python.org/3/
→ Used for understanding core Python concepts and syntax.

2. pandas Documentation
https://pandas.pydata.org/docs/
→ Used for data manipulation, reading CSV files, and calculations.

3. Matplotlib Documentation
https://matplotlib.org/stable/contents.html
→ Used for generating graphs and visualizations.